

Core Language Working Group "tentatively ready" Issues for the March, 2018 (Jacksonville) meeting

Section references in this document reflect the section numbering of document [WG21 N4700](#).

1893. Function-style cast with *braced-init-lists* and empty pack expansions

Section: 8.2.3 [expr.type.conv] **Status:** tentatively ready **Submitter:** Richard Smith **Date:** 2014-03-13

According to 8.2.3 [expr.type.conv] paragraph 1,

A simple-type-specifier (10.1.7.2 [dcl.type.simple]) or *typename-specifier* (17.7 [temp.res]) followed by a parenthesized *expression-list* constructs a value of the specified type given the expression list. If the expression list is a single expression, the type conversion expression is equivalent (in definedness, and if defined in meaning) to the corresponding cast expression (8.4 [expr.cast]). If the type specified is a class type, the class type shall be complete. If the expression list specifies more than a single value, the type shall be a class with a suitably declared constructor (11.6 [dcl.init], 15.1 [class.ctor]), and the expression $\tau(x_1, x_2, \dots)$ is equivalent in effect to the declaration $\tau \text{ } t(x_1, x_2, \dots)$; for some invented temporary variable t , with the result being the value of t as a prvalue.

This does not cover the cases when the *expression-list* contains a single *braced-init-list* (which is neither an expression nor more than a single value) or if it contains no expressions as the result of an empty pack expansion.

Proposed resolution (June, 2014): [SUPERSEDED]

This issue is resolved by the resolution of issue 1299.

Proposed resolution (November, 2017)

Change 8.2.3 [expr.type.conv] paragraph 2 as follows:

If the initializer is a parenthesized single expression, the type conversion expression is equivalent to the corresponding cast expression (8.4 [expr.cast]). Otherwise, if the type is cv void and the initializer is () **(after pack expansion, if any)**, the expression is a prvalue of the specified type that performs no initialization. Otherwise, the expression is a prvalue of the specified type whose result object is direct-initialized (11.6 [dcl.init]) with the initializer. ~~For an expression of the form $\tau()$, τ~~ **If the initializer is a parenthesized optional expression-list, the specified type** shall not be an array type.

1910. "Shall" requirement applied to runtime behavior

According to 6.7.4.1 [basic.stc.dynamic.allocation] paragraph 3,

If an allocation function declared with a non-throwing *exception-specification* (18.4 [except.spec]) fails to allocate storage, it shall return a null pointer. Any other allocation function that fails to allocate storage shall indicate failure only by throwing an exception (18.1 [except.throw]) of a type that would match a handler (18.3 [except.handle]) of type `std::bad_alloc` (21.6.3.1 [bad.alloc]).

The use of the word “shall” to constrain runtime behavior is inappropriate, as it normally identifies cases requiring a compile-time diagnostic.

Proposed resolution (November, 2017)

1. Change 6.7.4 [basic.stc.dynamic] paragraph 3 as follows:

~~Any allocation and/or deallocation functions defined in a C++ program, including the default versions in the library, shall conform to the semantics~~ **If the behavior of an allocation or deallocation function does not satisfy the semantic constraints** specified in 6.7.4.1 [basic.stc.dynamic.allocation] and 6.7.4.2 [basic.stc.dynamic.deallocation], **the behavior is undefined.**

2. Change 6.7.4.1 [basic.stc.dynamic.allocation] paragraph 1 as follows:

...The value of the first parameter ~~shall be~~ **is** interpreted as the requested size of the allocation...

3. Change 6.7.4.1 [basic.stc.dynamic.allocation] paragraph 2 as follows:

~~The~~ **An** allocation function attempts to allocate the requested amount of storage. If it is successful, it ~~shall return~~ **returns** the address of the start of a block of storage whose length in bytes ~~shall be~~ **is** at least as large as the requested size. ~~There are no constraints on the contents of the allocated storage on return from the allocation function.~~ The order, contiguity, and initial value of storage allocated by successive calls to an allocation function are unspecified. ~~The~~ **For an allocation function other than a reserved placement allocation function (21.6.2.3 [new.delete.placement], the** pointer returned ~~shall be~~ **is** suitably aligned so that it can be converted to a pointer to any suitable complete object type (21.6.2.1 [new.delete.single]) and then used to access the object or array in the storage allocated (until the storage is explicitly deallocated by a call to a corresponding deallocation function). Even if the size of the space requested is zero, the request can fail. If the request succeeds, the value returned ~~shall be~~ **by a replaceable allocation function is** a non-null pointer value (7.11 [conv.ptr]) `p0` different from any previously returned value `p1`, unless that value `p1` was subsequently passed to ~~an operator delete a~~ **replaceable deallocation function**. Furthermore, for the library allocation functions in 21.6.2.1 [new.delete.single] and 21.6.2.2 [new.delete.array], `p0` ~~shall represent~~ **represents** the address of a block of storage disjoint from the storage for any other object accessible to the caller. The effect of indirecting through a pointer returned ~~as~~ **from** a request for zero size is undefined.³⁸

4. Change 6.7.4.1 [basic.stc.dynamic.allocation] paragraph 3 as follows:

An allocation function that fails to allocate storage can invoke the currently installed new-handler function (21.6.3.3 [new.handler]), if any. [Note: A program-supplied allocation function can obtain the address of the currently installed `new_handler` using the `std::get_new_handler` function (21.6.3.4 [set.new.handler]). —end note] ~~If an~~ **An** allocation function that has a non-throwing exception specification (18.4 [except.spec]) ~~fails to allocate storage, it shall return~~ **indicates failure by returning** a null pointer **value**. Any other allocation function ~~that fails to allocate storage shall indicate~~ **never returns a null pointer value and indicates** failure only by throwing an exception (18.1 [except.throw]) of a type that would match a handler (18.3 [except.handle]) of type `std::bad_alloc` (21.6.3.1 [bad.alloc]).

1983. Inappropriate use of *virt-specifier*

Section: 12.2 [class.mem] **Status:** tentatively ready **Submitter:** Richard Smith **Date:** 2014-08-11

The restriction in 12.2 [class.mem] paragraph 8 that a *virt-specifier* may appear only in the declaration of a virtual function is insufficient to rule out examples like the following:

```
struct A { virtual void f(); };
struct B { friend void A::f() final; };

template<typename T> struct C { virtual void f() {} };
template void C<int>::f() final;
template<> void C<char>::f() final;
```

One possibility might be to require that a *virt-specifier* appear only on the first declaration of a function.

Proposed resolution (November, 2017)

Change 12.2 [class.mem] paragraph 13 as follows:

A *virt-specifier-seq* shall contain at most one of each *virt-specifier*. A *virt-specifier-seq* shall appear only in the **first** declaration of a virtual member function (13.3 [class.virtual]).

2059. Linkage and deduced return types

Section: 10.1.7.4 [dcl.spec.auto] **Status:** tentatively ready **Submitter:** Richard Smith **Date:** 2014-12-15

Use of function return type deduction makes it possible to define functions whose return type is a type without linkage. Although 6.5 [basic.link] paragraph 8 permits such a usage if the function is defined in the same translation unit as it is used, it may be helpful to consider changing the overall rules regarding the use of types with internal or no linkage. As an example, the following example permits access to a local static variable that has not been initialized:

```
auto f() {
    static int n = 123;
    struct X { int &f() { return n; } };
    return X();
}
int &r = decltype(f())().f();
```

Notes from the February, 2016 meeting:

CWG agreed that the current rule in 6.5 [basic.link] paragraph 8 is unneeded; the ODR already prohibits use of an entity that is not defined in the current translation unit and cannot be defined in a different translation unit.

Proposed resolution (November, 2017)

Change 6.5 [basic.link] paragraph 8 as follows:

~~...A type without linkage shall not be used as the type of a variable or function with external linkage unless~~

- ~~• the entity has C language linkage (10.5 [dcl.link]), or~~
- ~~• the entity is declared within an unnamed namespace (10.3.1 [namespace.def]), or~~
- ~~• the entity is not odr-used (6.2 [basic.def.odr]) or is defined in the same translation unit.~~

[Note: In other words, a type without linkage contains a class or enumeration that cannot be named outside its translation unit. ~~An entity with external linkage declared using such a type could not correspond to any~~

other entity in another translation unit of the program and thus must be defined in the translation unit if it is odr-used. Also note that classes **Classes** with linkage may contain members whose types do not have linkage, and that typedef. **Typedef** names are ignored in the determination of whether a type has linkage. —end note]
[Example:

```
template <class T> struct B {
    void g(T) { }
    void h(T);
    friend void i(B, T) { }
};

void f() {
    struct A { int x; }; // no linkage
    A a = { 1 };
    B<A> ba;              // declares B<A>::g(A) and B<A>::h(A)
    ba.g(a);              // OK
    ba.h(a);              // error: B<A>::h(A) not defined; A cannot be named in the another translation unit
    i(ba, a);             // OK
}
```

—end example]

2081. Deduced return type in redeclaration or specialization of function template

Section: 10.1.7.4 [dcl.spec.auto] **Status:** tentatively ready **Submitter:** John Spicer **Date:** 2015-02-05

10.1.7.4 [dcl.spec.auto] paragraph 13 says,

Redeclarations or specializations of a function or function template with a declared return type that uses a placeholder type shall also use that placeholder, not a deduced type.

The inverse should also be true (a specialization cannot use a placeholder type if the template used a non-placeholder), but this is not said explicitly.

Proposed resolution (November, 2017)

Change 10.1.7.4 [dcl.spec.auto] paragraph 11 as follows:

Redeclarations or specializations of a function or function template with a declared return type that uses a placeholder type shall also use that placeholder, not a deduced type. **Similarly, redeclarations or specializations of a function or function template with a declared return type that does not use a placeholder type shall not use a placeholder.** [Example:

2088. Late tiebreakers in partial ordering

Section: 17.9.2.4 [temp.deduct.partial] **Status:** tentatively ready **Submitter:** Richard Smith **Date:** 2015-02-19

The late tiebreakers for lvalue-vs-rvalue references and cv-qualification in 17.9.2.4 [temp.deduct.partial] paragraph 9 are applied

If, for a given type, deduction succeeds in both directions (i.e., the types are identical after the transformations above) and both P and A were reference types (before being replaced with the type referred to above):

However, this is based on a false assumption. For example,

```
template <typename T> struct A {
    struct typeA { };
    struct typeB { };
    using convTyA = T (*const &&)(typename A<T>::typeA);
    using convTyB = T (*const &)(typename A<T>::typeB);
    operator convTyA();
    operator convTyB();
};

template <typename T> void foo(T (*const &&)(typename A<T>::typeA));
template <typename T> int foo(T (*const &)(typename A<T>::typeB));

int main() {
    return foo<int>(A<int>());
}
```

(see also issues 1847 and 1157.). We need to decide whether the rule is “deduction succeeds in both directions” or “the types are identical.” The latter seems more reasonable.

Proposed resolution (November, 2017)

Change 17.9.2.4 [temp.deduct.partial] paragraph 9 as follows:

If, for a given type, ~~deduction succeeds in both directions (i.e.,~~ the types are identical after the transformations above) and both P and A were reference types (before being replaced with the type referred to above):

- if the type from the argument template was an lvalue reference and the type from the parameter template was not, the parameter type is not considered to be at least as specialized as the argument type; otherwise,
- if the type from the argument template is more cv-qualified than the type from the parameter template (as described above), the parameter type is not considered to be at least as specialized as the argument type.

2092. Deduction failure and overload resolution

Section: 17.9.3 [temp.over] **Status:** tentatively ready **Submitter:** Fedor Sergeev **Date:** 2015-03-06

Given an example like

```
template <class T = int> void foo(T*);

void test()
{
    foo(0);    // #1 valid?
    foo<>(0);  // #2 valid?
}
```

most/all implementations reject this code. However, the wording of the Standard only invokes 17.9.3 [temp.over] (“Overload resolution”) in cases where there is more than one function or function template, which is not the case here. The current wording would appear to make this well-formed because of the application of 17.9.1 [temp.arg.explicit] paragraph 2. Perhaps overload resolution should apply even when there is a single function template?

Notes from the May, 2015 meeting:

This issue is mostly a duplicate of issue 1582. However, CWG felt that it should be clarified that overload resolution applies in all cases, not just when templates are overloaded, so the issue is being left open to deal with that aspect.

Proposed resolution (November, 2017)

1. Change 8.2.2 [expr.call] paragraph 1, splitting it into three paragraphs, as follows:

A function call is a postfix expression followed by parentheses containing a possibly empty, comma-separated list of *initializer-clauses* which constitute the arguments to the function. The postfix expression shall have function type or function pointer type. For a call to a non-member function or to a static member function, the postfix expression shall be either an lvalue that refers to a function (in which case the function-to-pointer standard conversion (7.3 [conv.func]) is suppressed on the postfix expression), or it shall have function pointer type. **Calling a function through an expression whose function type is different from the function type of the called function's definition results in undefined behavior (10.5 [dcl.link]).**

For a call to a non-static member function, the postfix expression shall be an implicit (12.2.2 [class.mfct.non-static], 12.2.3 [class.static]) or explicit class member access (8.2.5 [expr.ref]) whose *id-expression* is a function member name, or a pointer-to-member expression (8.5 [expr.mptr.oper]) selecting a function member; the call is as a member of the class object referred to by the object expression. In the case of an implicit class member access, the implied object is the one pointed to by this. [Note: A member function call of the form `f()` is interpreted as `(*this).f()` (see 12.2.2 [class.mfct.non-static]). —end note]

If a function or member function name is used, **the name can be overloaded (Clause 16 [over]), in which case the appropriate function shall be selected and the validity of the call are determined** according to the rules in 16.3 [over.match]. If the selected function is non-virtual, or if the *id-expression* in the class member access expression is a qualified-id, that function is called. Otherwise, its final overrider (13.3 [class.virtual]) in the dynamic type of the object expression is called; such a call is referred to as a *virtual function call*. [Note: The dynamic type is the type of the object referred to by the current value of the object expression. 15.7 [class.ctor] describes the behavior of virtual function calls when the object expression refers to an object under construction or destruction. —end note]

2. Add the following to 8.2.2 [expr.call] as a new paragraph before the existing paragraph 4:

Calling a function through an expression whose function type is different from the function type of the called function's definition results in undefined behavior (10.5 [dcl.link]).

When a function is called, each parameter (11.3.5 [dcl.fct]) shall be initialized...

3. Change 16 [over] paragraph 2 as follows:

When **an overloaded** a function name is used in a call, which **overloaded** function declaration is being referenced **is and the validity of the call are** determined by comparing the types of the arguments at the point of use with the types of the parameters in the **overloaded** declarations that are visible at the point of use. This function selection process is called *overload resolution*...

4. Change 17.9.3 [temp.over] paragraph 1 as follows:

A function template can be overloaded either by (non-template) functions of its name or by (other) function templates of the same name. When a call to **that** **the name of a function or function template** is written (explicitly, or implicitly using the operator notation), template argument deduction...

Consider the following example:

```
const int i = -1;
namespace T {
    namespace N { const int i = 1; }
    namespace M {
        using namespace N;
        int a[i];
    }
}
```

According to 6.4.1 [basic.lookup.unqual], lookup for `i` finds `T::N::i` and stops. However, according to 6.3.10 [basic.scope.hiding] paragraph 1, the appearance of `T::N::i` in namespace `T` does *not* hide `::i`, so both declarations of `i` are visible in the declaration of `a`.

It seems strange that we specify this name hiding rule in two different ways in two different places, but they should at least be consistent.

On a related note, the wording in 6.4.1 [basic.lookup.unqual] paragraph 2, “as if they were declared in the nearest enclosing namespace...” could be confusing with regard to the “declared in the same scope” provisions of 6.3.10 [basic.scope.hiding].

Proposed resolution (November, 2017)

Change 6.3.10 [basic.scope.hiding] paragraphs 1 and 2 as follows:

~~A name can be hidden by an explicit declaration of that same name in a nested declarative region or derived class (13.2 [class.member.lookup]).~~ **A declaration of a name in a nested declarative region hides a declaration of the same name in an enclosing declarative region; see 6.3.1 [basic.scope.declarative] and 6.4.1 [basic.lookup.unqual].**

~~A class name (12.1 [class.name]) or enumeration name (10.2 [dcl.enum]) can be hidden by the name of a variable, data member, function, or enumerator declared in the same scope. If a class name (12.1 [class.name]) or enumeration name (10.2 [dcl.enum]) and a variable, data member, function, or enumerator are declared in the same scope declarative region (in any order) with the same name (excluding declarations made visible via using-directives (6.4.1 [basic.lookup.unqual])), the class or enumeration name is hidden wherever the variable, data member, function, or enumerator name is visible.~~

2226. Xvalues vs lvalues in conditional expressions

In the following example,

```
const T a;
T b;
false ? a : std::move(b);
```

the most appropriate result would seem to be that the expression is an lvalue of type `const T` that refers to either `a` or `b`. However, because 8.16 [expr.cond] bullet 4.1 requires that the conversion bind directly to an lvalue, while `std::move(b)` is an xvalue, the result is a `const T` temporary copy-initialized from `std::move(b)`.

Proposed resolution (November, 2017)

Change 8.16 [expr.cond] bullet 4.1 as follows:

Attempts are made to form an implicit conversion sequence from an operand expression E1 of type T1 to a target type related to the type T2 of the operand expression E2 as follows:

- If E2 is an lvalue, the target type is “lvalue reference to T2”, subject to the constraint that in the conversion the reference must bind directly (11.6.3 [dcl.init.ref]) to **an lvalue a glvalue**.
- ...

2227. Destructor access and default member initializers

Section: 15.6.2 [class.base.init] **Status:** tentatively ready **Submitter:** Richard Smith **Date:** 2016-02-01

There is implementation divergence on the validity of the following:

```
class X { ~X(); };  
struct Y { X x = {}; };
```

Should x's destructor be potentially invoked by this attempt to initialize an x object? Or,

```
auto *y = new Y {};
```

No constructor for Y is used, because this is aggregate initialization, and a destructor for X is not strictly necessary as there is no later initialization that might throw, but in the corresponding default constructor case we do require that the destructor be valid.

Perhaps the most consistent answer is that the default member initializer should not potentially invoke the destructor unless it's used (for symmetry with default arguments), but that aggregate initialization should potentially invoke the destructors of all subobjects (including the final one - exceptions could theoretically be thrown between the completion of the construction of the final aggregate element and the notional completion of the construction of the aggregate itself.

Proposed resolution (November, 2017)

1. Add the following as a new paragraph following 11.6.1 [dcl.init.aggr] paragraph 7:

An aggregate that is a class can also be initialized with a single expression not enclosed in braces, as described in 11.6 [dcl.init].

The destructor for each element of class type is potentially invoked (15.4 [class.dtor]) from the context where the aggregate initialization occurs. [Note: This provision ensures that destructors can be called for fully-constructed subobjects in case an exception is thrown (18.2 [except.ctor]). —end note]

2. Change 15.4 [class.dtor] paragraph 12 as follows:

...A destructor is *potentially invoked* if it is invoked or as specified in 8.3.4 [expr.new], **11.6.1 [dcl.init.aggr]**, 15.6.2 [class.base.init], and 18.1 [except.throw]. A program is ill-formed if a destructor that is potentially invoked is deleted or not accessible from the context of the invocation.

2229. Volatile unnamed bit-fields

Section: 12.2.4 [class.bit] **Status:** tentatively ready **Submitter:** David Majnemer **Date:** 2016-02-08

According to 12.2.4 [class.bit] paragraph 2, unnamed bit-fields are not members, but there does not appear to be a prohibition against their being declared `volatile`. Is this intended?

Proposed resolution (November, 2017)

Change 12.2.4 [class.bit] paragraph 2 as follows:

A declaration for a bit-field that omits the *identifier* declares an *unnamed bit-field*. Unnamed bit-fields are not members and cannot be initialized. **An unnamed bit-field shall not be declared with a cv-qualified type.** [Note: An unnamed bit-field is useful for padding...

2234. Missing rules for *simple-template-id* as *class-name*

Section: 12 [class] **Status:** tentatively ready **Submitter:** Richard Smith **Date:** 2016-02-25

There does not seem to be a rule that prohibits an example like:

```
template<typename T> struct X;
struct X<int> {
};
```

Proposed resolution (November, 2017)

Change 12 [class] paragraph 1 as follows:

...**A class declaration where the *class-name* in the *class-head-name* is a *simple-template-id* shall be an explicit specialization (17.8.3 [temp.expl.spec]) or a partial specialization (17.6.5 [temp.class.spec]).** A *class-specifier* whose *class-head* omits the *class-head-name* defines an unnamed class. [Note: An unnamed class thus can't be final. —end note]

2235. Partial ordering and non-dependent types

Section: 17.9.2.4 [temp.deduct.partial] **Status:** tentatively ready **Submitter:** Richard Smith **Date:** 2016-02-25

Paragraph 12 of 17.9.2.4 [temp.deduct.partial] contains the following example:

```
template <class T> T f(int);           // #1
template <class T, class U> T f(U);    // #2
void g() {
    f<int>(1);                         // calls #1
}
```

However, paragraph 4 states,

If a particular *P* contains no *template-parameters* that participate in template argument deduction, that *P* is not used to determine the ordering.

Thus, we ignore the *P*=*int*, *A*=*U* case and deduction succeeds for the *P*=*U*, *A*=*int* case, so both templates are at least as specialized as each other. And consider:

```
template <class... T> struct V {};
template <class... Ts, class... Us> void Foo(V<Ts...>, V<Us&...>) {} // #3
template <class... Us> void Foo(V<>, V<Us&...>) {}                  // #4
```

```
void h() {
    Foo(V<>(), V<>());
}
```

The intent is that this should call #4; that template clearly ought to be more specialized.

Proposed resolution (November, 2017)

1. Change 17.9.2.4 [temp.deduct.partial] paragraph 4 as follows:

:

Each type nominated above from the parameter template and the corresponding type from the argument template are used as the types of P and A. ~~If a particular P contains no template parameters that participate in template argument deduction, that P is not used to determine the ordering.~~

2. Change 17.9.2.5 [temp.deduct.type] paragraph 4 as follows:

...If a template parameter is used only in non-deduced contexts and is not explicitly specified, template argument deduction fails. [Note: Under 17.9.2.1 [temp.deduct.call] ~~and 17.9.2.4 [temp.deduct.partial]~~, if P contains no *template-parameters* that appear in deduced contexts, no deduction is done, so P and A need not have the same form. —end note]

2237. Can a *template-id* name a constructor?

Section: 15.1 [class.ctor] **Status:** tentatively ready **Submitter:** Faisal Vali **Date:** 2016-03-02

Bullet 1.2 of 15.1 [class.ctor], describing declarator forms that are considered to declare a constructor, says:

...and the *id-expression* has one of the following forms:

- ...
- in a *member-declaration* that belongs to the *member-specification* of a class template but is not a friend declaration, the *id-expression* is a *class-name* that names the current instantiation (17.7.2.1 [temp.dep.type]) of the immediately-enclosing class template; or
- ...

The term *class-name* includes *simple-template-id*. It is not clear that allowing a constructor declaration of the form

```
template<class T> struct X {
    X<T>(T); // constructor
};
```

is useful or helpful.

Proposed resolution (November, 2017)

1. Change 15.1 [class.ctor] paragraph 1 as follows:

...and the *id-expression* has one of the following forms:

- in a *member-declaration* that belongs to the *member-specification* of a class **or class template** but is not a friend declaration (14.3 [class.friend]), the *id-expression* is the injected-class-name (Clause 12 [class]) of the immediately-enclosing ~~class~~ **entity or**

- ~~in a member-declaration that belongs to the member-specification of a class template but is not a friend declaration, the id-expression is a class-name that names the current instantiation (17.7.2.1 [temp.dep.type]) of the immediately-enclosing class template; or~~
- in a declaration at namespace scope or in a friend declaration, the *id-expression* is a *qualified-id* that names a constructor (6.4.3.1 [class.qual]).

2. Change 15.4 [class.dtor] paragraph 1 as follows:

...and the *id-expression* has one of the following forms:

- in a *member-declaration* that belongs to the *member-specification* of a class **or class template** but is not a friend declaration (14.3 [class.friend]), the *id-expression* is *~class-name* and the *class-name* is the injected-class-name (Clause 12 [class]) of the immediately-enclosing **class;** **entity or**
- ~~in a member-declaration that belongs to the member-specification of a class template but is not a friend declaration, the id-expression is ~class-name and the class-name names the current instantiation (17.7.2.1 [temp.dep.type]) of the immediately-enclosing class template; or~~
- in a declaration at namespace scope or in a friend declaration, the *id-expression* is *nested-name-specifier ~class-name* and the *class-name* names the same class as the *nested-name-specifier*.

3. Add the following as a new paragraph in C.5 [diff.cpp17]:

C.5.x Clause 15: Special member functions [diff.cpp17.special]

Affected subclauses: 15.1 [class.ctor], 15.4 [class.dtor]

Change: A *simple-template-id* is no longer valid as the *declarator-id* of a constructor or destructor.

Rationale: Remove potentially error-prone option for redundancy.

Effect on original feature: Valid C++ 2017 code may fail to compile.

```
template<class T>
struct A {
    A<T>(); // error: simple-template-id not allowed for constructor
    A(int); // OK, injected-class-name used
    ~A<T>(); // error: simple-template-id not allowed for destructor
};
```

(Note that this resolution is a change for C++20, NOT a defect report against C++17 and earlier versions.)

2255. Instantiated static data member templates

Section: 17.8 [temp.spec] **Status:** tentatively ready **Submitter:** Mike Miller **Date:** 2016-03-29

The current wording does not state that a specialization of a static data member template (17 [temp] paragraph 1) is a static data member, which leaves the status of an example like the following unclear (since 8.2.5 [expr.ref] bullet 4.1 is phrased in terms of static data members):

```
template <class T> struct A {
    template <class U> static const U x = 1;
    static const int y = 2;
};

int main() {
    A<int> a;
    int y = a.y;           // OK
```

```
int x = a.x<int>;    // ???
}
```

Proposed resolution (November, 2017)

Change 17.8 [temp.spec] paragraph 2 as follows:

A function instantiated from a function template is called an instantiated function. A class instantiated from a class template is called an instantiated class. A member function, a member class, a member enumeration, or a static data member of a class template instantiated from the member definition of the class template is called, respectively, an instantiated member function, member class, member enumeration, or static data member. A member function instantiated from a member function template is called an instantiated member function. A member class instantiated from a member class template is called an instantiated member class. **A variable instantiated from a variable template is called an instantiated variable. A static data member instantiated from a static data member template is called an instantiated static data member.**

2260. Explicit specializations of deleted member functions

Section: 17.8.3 [temp.expl.spec] **Status:** tentatively ready **Submitter:** Richard Smith **Date:** 2016-04-17

Although the Standard allows for explicitly specializing a deleted function template, member function of a class template, or member function template with a non-deleted definition, this seems to be problematic for non-template member functions of class templates. For example:

```
template<typename T> struct A {
    A(const A&) = delete;
    A(A&&) = default;
};
static_assert(is_trivially_copyable(A<int>));
template<> struct A<int>::A(const A&) { /* ... */ }
static_assert(is_trivially_copyable(A<int>));
template<typename T> struct B {
    virtual void f() = delete;
};
struct C : B<int> { void f() override = delete; }; // ok, overriding deleted with deleted
template<> void B<int>::f() {} // would make C retroactively ill-formed?
```

Notes from the December, 2016 teleconference:

=delete definitions of member functions should be instantiated when instantiating a class template. That would make the example an ill-formed redefinition.

Proposed resolution (November, 2017)

Change 17.8.1 [temp.inst] paragraph 2, breaking the running text into bullets, as follows:

The implicit instantiation of a class template specialization causes

- the implicit instantiation of the declarations, but not of the definitions, ~~default arguments, or noexcept-specifiers~~ of the **non-deleted** class member functions, member classes, scoped member enumerations, static data members, member templates, and friends; and
- ~~it causes~~ the implicit instantiation of the definitions of **deleted member functions**, unscoped member enumerations, and member anonymous unions.

The implicit instantiation of a class template specialization does not cause the implicit instantiation of default arguments or noexcept-specifiers of the class member functions. [Example:

```

template<class T>
struct C {
    void f() { T x; }
    void g() = delete;
};
C<void> c; // OK, definition of C<void>::f is not instantiated at this point
template<> void C<int>::g() { } // error: redefinition of C<int>::g

```

—**end example** However, for the purpose of determining whether an instantiated redeclaration is valid according to 6.2 [basic.def.odr] and 12.2 [class.mem], a declaration that corresponds to a definition in the template is considered to be a definition. [Example:

2299. constexpr vararg functions

Section: 10.1.5 [dcl.constexpr] **Status:** tentatively ready **Submitter:** Daveed Vandevor **Date:** 2016-04-11

It is not clear whether a constexpr function can be a vararg function or not. In particular, it is unclear if `va_list` is a literal type and whether `va_start`, `va_arg`, and `va_end` produce constant expressions.

Proposed resolution (November, 2017)

1. Add a new bullet to the list in 8.20 [expr.const] paragraph 2, and update the text as follows:

An expression *e* is a core constant expression unless the evaluation of *e*, following the rules of the abstract machine (4.6 [intro.execution]), would evaluate one of the following expressions:

- ...
- a relational (8.9 [expr.rel]) or equality (8.10 [expr.eq]) operator where the result is unspecified;
or
- a *throw-expression* (8.17 [expr.throw]); or
- an invocation of the `va_arg` macro (21.10.1 [cstdarg.syn]).

If *e* satisfies the constraints of a core constant expression, but evaluation of *e* would evaluate an operation that has undefined behavior as specified in Clause 20 [library] through Clause 33 [thread] of this document, or an invocation of the `va_start` macro (21.10.1 [cstdarg.syn]), it is unspecified whether *e* is a core constant expression.